```python
import random


# Konstanten
ANZAHL_SPIELE = 100000
ANZAHL_KARTEN = 5
ANZAHL_KARTEN_DECK = 52
ANZAHL_KARTEN_WERTE = 13
KARTEN_WERT_START = 2
KARTEN_WERT_ASS = 14
ANZAHL_KARTEN_HAND = 5
ANZAHL_FARBEN = 4
ANZAHL_VIERER = 4
ANZAHL_DREIER = 3
ANZAHL_ZWEIER = 2
ANZAHL_EINER = 1
PROZENT_FAKTOR = 100
AUSGABE_NACHKOMMA = 5
AUSGABE_BREITE = 40

ROYAL_FLUSH_WERTE = [10, 11, 12, 13, 14]
ASS_STRASSE_WERTE = [2, 3, 4, 5, 14]
STRASSEN_ABSTAND = 4

KOMBINATIONEN = [
    "Royal Flush",
    "Straight Flush",
    "Poker",
    "Full House",
    "Flush",
    "Strasse",
    "Drilling",
    "Zwei Paare",
    "Ein Paar",
    "High Card"
]


# Zieht fünf unterschiedliche Karten
# wird in einer liste gespeichert
def ziehe_Karten() -> list[int]:
    # sample heißt dass nichts doppelt gezogen wird
    return random.sample(
        range(ANZAHL_KARTEN_DECK),
        ANZAHL_KARTEN
    )

# die karten werden aus der hand
# genommen und farben und werte werden ausgerechnet
# mit tuple können wir 2 listen in einem zurückgeben, sonst
# 2 extra Methoden, tupel ist nicht veränderbar
def erstelle_Karten_Daten(hand: list[int] ) -> tuple[list[int], list[int]]:
    werte = []
    farben = []

    for karte in hand:
        # wenn zB 11 % 13 = 11 (rest) also ist wert 13
        werte.append( karte % ANZAHL_KARTEN_WERTE + KARTEN_WERT_START )
        # wenn man < 12 / 13 immer 0, über 12 = 1 usw. jede Zahl eine Farbe
        farben.append(karte // ANZAHL_KARTEN_WERTE)

    return werte, farben
```

```python
# Zählt die Häufigkeit der Kartenwerte
def zaehle_Werte(werte: list[int]) -> list[int]:
    anzahl = []

    # bei zB [7, 7, 2, 2, 2] -> [7, 2] dass es nicht doppelt gezählt wird
    for wert in set(werte):
        anzahl.append(werte.count(wert))
    # sortieren  also [2, 3]
    return sorted(anzahl, reverse=True)


# Prüft, ob alle Karten dieselbe Farbe haben
def ist_Flush(farben: list[int]) -> bool:
    # set entfernt doppelte, wenn alle gleiche Farbe haben
    return len(set(farben)) == ANZAHL_EINER


# Prüft, ob die Karten eine Strasse bilden
def ist_Strasse(werte: list[int]) -> bool:
    sortierteWerte = sorted(werte)

    # wenn nt alle verschieden sind kanns keine Straße sein
    if len(set(werte)) != ANZAHL_KARTEN_HAND:
        return False

    if sortierteWerte == ASS_STRASSE_WERTE:
        return True
    # oder wenn letzter - erster Wert Differenz 4 ist
    return (
        sortierteWerte[-1] - sortierteWerte[0]
        == STRASSEN_ABSTAND
    )


# Prüft auf Royal Flush
def ist_Royal_Flush( werte: list[int], farben: list[int]) -> bool:
    # wenn selbe Farben und werte von Royal Flush
    return (
        ist_Flush(farben)
        and sorted(werte) == ROYAL_FLUSH_WERTE
    )


# Prüft auf Straight Flush
def ist_Straight_Flush(werte: list[int],farben: list[int]) -> bool:
    return (
        ist_Flush(farben)
        and ist_Strasse(werte)
        and not ist_Royal_Flush(werte, farben)
    )


# Prüft auf Poker (vier gleiche Karten)
def ist_Poker(anzahl: list[int]) -> bool:
    return anzahl == [ANZAHL_VIERER, ANZAHL_EINER]


# Prüft auf Full House
def ist_Full_House(anzahl: list[int]) -> bool:
    return anzahl == [ANZAHL_DREIER, ANZAHL_ZWEIER]
```

```python
# Prüft auf Drilling
def ist_Drilling(anzahl: list[int]) -> bool:
    return anzahl == [
        ANZAHL_DREIER,
        ANZAHL_EINER,
        ANZAHL_EINER
    ]


# Prüft auf zwei Paare
def ist_Zwei_Paare(anzahl: list[int]) -> bool:
    return anzahl == [
        ANZAHL_ZWEIER,
        ANZAHL_ZWEIER,
        ANZAHL_EINER
    ]


# Prüft auf ein Paar
def ist_Ein_Paar(anzahl: list[int]) -> bool:
    return anzahl == [
        ANZAHL_ZWEIER,
        ANZAHL_EINER,
        ANZAHL_EINER,
        ANZAHL_EINER
    ]


# Bestimmt die stärkste Pokerkombination
def pruefe_Kombination(hand: list[int]) -> str:
    # zuerst hol ich die gezogenen karten und speicher sie ab
    werte, farben = erstelle_Karten_Daten(hand)
    # dann die anzahl der doppelten zahlen
    anzahl = zaehle_Werte(werte)

    if ist_Royal_Flush(werte, farben):
        return "Royal Flush"

    if ist_Straight_Flush(werte, farben):
        return "Straight Flush"

    if ist_Poker(anzahl):
        return "Poker"

    if ist_Full_House(anzahl):
        return "Full House"

    if ist_Flush(farben):
        return "Flush"

    if ist_Strasse(werte):
        return "Strasse"

    if ist_Drilling(anzahl):
        return "Drilling"

    if ist_Zwei_Paare(anzahl):
        return "Zwei Paare"

    if ist_Ein_Paar(anzahl):
        return "Ein Paar"
```

```
    return "High Card"


# Führt die Simulation durch
def simuliere_Poker() -> dict[str, int]:
    ergebnisse = {}

    for kombination in KOMBINATIONEN:
        # leeres dictionary, jede kombination bis jz 0x
        ergebnisse[kombination] = 0

    for spiel in range(ANZAHL_SPIELE):
        # karte ziehen
        hand = ziehe_Karten()
        #kombi speichern
        kombination = pruefe_Kombination(hand)

        ergebnisse[kombination] += 1

    return ergebnisse


# dictionary wird übergeben
def zeige_Ergebnisse(
    ergebnisse: dict[str, int]
) -> None:
    print("POKER-SIMULATION")
    print("================")
    print("Anzahl Spiele:", ANZAHL_SPIELE)
    print()

    print(
        f"{'Kombination':<18}"
        f"{'Anzahl':>10}"
        f"{'Prozent':>12}"
    )
    # trennlinie
    print("-" * AUSGABE_BREITE)

    # kombination zeile für zeile ausgeben
    for kombination in KOMBINATIONEN:
        anzahl = ergebnisse[kombination]

        prozent = (
            anzahl / ANZAHL_SPIELE * PROZENT_FAKTOR
        )

        print(
            f"{kombination:<18}"
            f"{anzahl:>10}"
            f"{prozent:>11.{AUSGABE_NACHKOMMA}f} %"
        )


# Hauptprogramm
def main() -> None:
    ergebnisse = simuliere_Poker()
    zeige_Ergebnisse(ergebnisse)


# startet dann programm
main()
```